10/10 08 Testing de Software (debrief)

Resumen Ejecutivo

TÍTULO: La Prueba de Software: Un Proceso Destructivo con Aporte Constructivo

El **Testing de Software** es un proceso destructivo que busca intencionalmente defectos, asumiendo su presencia en el código. Un test exitoso es aquel que los encuentra, contribuyendo a la **confianza** en el producto y a reducir costos de corrección. El testing se enmarca en la disciplina de **Control de Calidad (QC)**, mientras que el **Aseguramiento de Calidad (QA)** se enfoca en la prevención a lo largo de todo el ciclo de vida.

Principios Fundamentales y Riesgos

La práctica se rige por principios como el **Testing Temprano**, que reduce drásticamente los costos de corrección, y el reconocimiento de que el **Testing Exhaustivo es Imposible**. Un riesgo constante es la **Paradoja del Pesticida**, que obliga a los *testers* a renovar sus casos de prueba para seguir encontrando defectos. El testing debe enfocarse en áreas de alto riesgo y donde se produce el **Agrupamiento de Defectos**.

Niveles y Estrategias

Las pruebas se llevan a cabo en niveles de granularidad creciente, siguiendo el **Modelo en V**:

1. **Unitario:** Realizado por desarrolladores, enfocado en componentes individuales (Clase/Método).
2. **Integración:** Verifica que los componentes trabajen juntos incrementalmente.
3. **Sistema:** Prueba el sistema completo, incluyendo **Requerimientos No Funcionales** (ej. *performance*, seguridad) en un ambiente lo más parecido a producción.
4. **Aceptación (Alfa/Beta):** Realizada por el usuario final para establecer confianza y validar que el sistema se ajusta a sus necesidades.

Las estrategias de **Caja Blanca** (basadas en la estructura interna, usando Complejidad Ciclomática y Cobertura) y **Caja Negra** (basadas en las especificaciones funcionales, usando Partición de Equivalencias y Valores Límites) se complementan para lograr una cobertura óptima.

Enfoques Modernos

En ambientes **Ágiles** (empíricos), el testing es una **actividad continua y paralela**, a diferencia del enfoque tradicional secuencial. La calidad es responsabilidad del equipo completo. Prácticas como **TDD** (*Test Driven Development*), donde se escriben las pruebas unitarias antes del código, aseguran la calidad desde el diseño y facilitan la **Automatización de Regresión**, esencial para la **Integración Continua** y la entrega frecuente de valor.

TESTING DE SOFTWARE o PRUEBA DE SOFTWARE

Síntesis: La prueba de software (o testing) es una disciplina fundamental dentro de la Ingeniería de Software, considerada una investigación técnica transversal. Su propósito es proveer información sobre la calidad del producto a los interesados (*stakeholders*). Aunque la prueba es un proceso **destructivo** que busca fallas, su aporte al proceso de desarrollo es **completamente constructivo**.

Explicación Clara y Vínculos con Calidad (QA vs. QC):

El testing forma parte de un esfuerzo más amplio para asegurar la calidad del software (Aseguramiento de Calidad de Software, o SQA).

- **Aseguramiento de Calidad (QA):** Es una actividad preventiva que se aplica a lo largo de **todo el proceso** de software, definiendo estándares y procedimientos que conducirán a obtener productos de alta calidad.

- **Control de Calidad (QC):** Es la aplicación de esos procesos de calidad para eliminar productos que no cumplen con el nivel requerido. **El testing es una función del Control de Calidad** que se lleva a cabo cuando el producto ya está construido.
- **Vínculo con el Contexto:** La calidad del software se logra y se inyecta a lo largo de todo el desarrollo; el testing se encarga de controlarla o revelarla, pero no la asegura por sí solo.

Definición de Prueba de Software

Definición Sólida: El testing es un **proceso diseñado para ejecutar un programa con la intención de encontrar errores o defectos**.

Desarrollo del Concepto (Actitud y Resultado):

1. **Actitud Destructiva:** La visión más apropiada del testing es que es un **proceso destructivo** de tratar de encontrar defectos, cuya presencia se asume en el código. El tester debe ir con una actitud negativa para demostrar que algo es incorrecto.
2. **Resultado Exitoso:** Un **testing exitoso es el que encuentra defectos**. Si no se encuentran fallos, se gana confianza en que el usuario encontrará menos problemas.
3. **Impacto Económico:** Mundialmente, el costo del testing para lograr un software confiable representa del **30% al 50% del costo total** del software.
4. **Verificación y Validación (V&V):** El testing ayuda a **verificar** (¿estamos construyendo el sistema correctamente?) y a **validar** (¿estamos construyendo el sistema correcto?).

Principios del Testing

Debido a que el tiempo y los recursos son limitados, la actividad de prueba debe ser dirigida por principios sólidos que optimicen el esfuerzo aplicado.

1. **El testing muestra presencia de defectos:** El testing solo puede demostrar la presencia de errores o defectos en un programa, pero no puede garantizar su ausencia. Si se encuentra un error, se revela un problema de calidad.
2. **El testing exhaustivo es imposible:** Probar cada secuencia posible de ejecución del programa es inviable. La decisión de cuánto probar depende de la evaluación del **nivel de riesgo** y los costos asociados.
3. **Testing temprano:** Las actividades de testing deberían comenzar lo antes posible, idealmente al mismo tiempo que se definen los requerimientos. Involucrar el testing temprano disminuye los costos de corrección de defectos.
4. **Agrupamiento de Defectos:** Los defectos en el software suelen agruparse en un **conjunto limitado de módulos**. Este principio ayuda a determinar dónde **testear primero** y a qué dedicarle **más esfuerzo**.
5. **Paradoja del Pesticida:** Si se utiliza el mismo conjunto de pruebas repetidamente, estas dejarán de ser efectivas para encontrar nuevos errores sutiles. Por lo tanto, los casos de prueba deben ser actualizados y rediseñados periódicamente.
6. **El testing es dependiente del contexto:** Los métodos y el esfuerzo de prueba deben adaptarse al contexto, considerando costos, riesgos y recursos específicos del proyecto.
7. **Falacia de la ausencia de errores:** Es un error suponer que el software está libre de fallos. La existencia de defectos es inevitable.
8. **El programador debería evitar probar su propio código:** Aunque el desarrollador realiza pruebas unitarias, la unidad de programación no debería probar sus propios desarrollos. Un equipo de prueba independiente (GPI) es aconsejable para evitar el conflicto de intereses inherente de querer demostrar que el código está libre de errores.

Mitos del Testing

Los mitos son ideas comunes pero incorrectas que deben ser corregidas para tener una visión madura del proceso de prueba.

- **Mito: El Testing es una etapa que comienza al terminar de codificar.**
  - **Realidad:** El testing debe ser una actividad **paralela y continua** al desarrollo, no una fase secuencial al final. El testing debe involucrarse tan temprano como la definición de requerimientos (testing temprano).
- **Mito: El Testing es probar que el software funciona.**
  - **Realidad:** El objetivo es **encontrar errores**. Un test exitoso es el que encuentra defectos. No se prueba para mostrar que funciona, sino con la suposición de que contiene defectos.
- **Mito: TESTING = CALIDAD de PRODUCTO o CALIDAD de PROCESO.**
  - **Realidad:** El testing **no asegura ni inyecta calidad**. Es un proceso de **control** de calidad que evalúa si la calidad que se incorporó durante el desarrollo es suficiente. La calidad se obtiene a lo largo de todo el proceso.
- **Mito: El tester es el enemigo del programador.**
  - **Realidad:** La búsqueda de fallas puede verse como una crítica, pero el *tester* es un **aliado y colaborador** cuyo aporte es constructivo para la calidad del producto.

Conceptos importantes

Error vs Defecto

- **Error (Error/Falla):** Es un fallo en un producto de trabajo. La manifestación de una falla en un producto.
- **Defecto (Bug):** Es la **falta verificada de conformidad con los requerimientos**. Un defecto es un error lógico, estructural u otro, que puede causar una falla operacional si se implementa tal como está especificado. Se presume su existencia en el código de antemano. Los bugs se esconden en las esquinas y se congregan en los límites.

Severidad y Prioridad

Estos dos conceptos son clave para el manejo de incidentes o defectos.

- **Severidad:** Define el **impacto** que tiene el defecto sobre el funcionamiento del sistema.
  - *Ejemplos de categorías:* Bloqueante, Crítico, Mayor, Menor, Cosmético.
- **Prioridad:** Define la **urgencia** con la que debe ser corregido el defecto.
  - *Ejemplos de categorías:* Urgencia, Alta, Media, Baja.

Caso de Prueba

Un **Caso de Prueba (CP)** es un conjunto de condiciones o variables bajo las cuales un *tester* determinará si el software está funcionando correctamente.

- **Objetivo:** Descubrir errores de forma completa con el mínimo esfuerzo y tiempo.
- **Buena práctica:** Una buena definición de casos de prueba ayuda a **reproducir los defectos**. Deben ser definidos de manera clara para que cualquier persona pueda interpretarlos sin ser un experto.
- **Derivación:** Los CP se derivan de la documentación del cliente (requerimientos, casos de uso) o de la información de relevamiento.

Ciclo de Test

Un ciclo de pruebas abarca la **ejecución de la totalidad de los casos de prueba** establecidos, aplicados a una misma versión del sistema a probar.

Regresión

La regresión es la verificación total de una nueva versión del sistema (después de que se han realizado cambios) para **prevenir la introducción de nuevos defectos** al intentar corregir o modificar el código existente.

- **Riesgo:** Siempre existe la probabilidad de que al arreglar un defecto, se introduzcan nuevos errores en otras partes del producto.
- **Buena Práctica:** Las pruebas de regresión deben realizarse cada vez que se hace un cambio importante al software, y la automatización es fundamental para reducir drásticamente los costos y el tiempo de este proceso.

NIVELES DE PRUEBA

Los niveles de prueba se realizan generalmente en un orden de granularidad inverso a como se desarrolla el software. Este concepto se ilustra en el **Modelo en V**.

Niveles de Testing: Testing Unitario

- **Síntesis:** El testing unitario se enfoca en verificar que los componentes individuales del código (ej. métodos, clases) funcionan correctamente de forma aislada.
- **Ejecución:**
  - Se prueba cada componente tras su realización/construcción.
  - Solo se prueban componentes individuales, de forma independiente.
  - Se realiza con **acceso al código bajo pruebas** (caja blanca).
  - Generalmente lo realizan los **desarrolladores**.
  - Se apoya en *frameworks* de pruebas unitarias (como JUnit) o herramientas de depuración.
  - Los errores se reparan tan pronto como se encuentran, sin constancia oficial de los incidentes.

Niveles de Testing: Testing de Integración

- **Síntesis:** Test orientado a verificar que las partes de un sistema que funcionan bien aisladamente, **también lo hacen en conjunto**.
- **Estrategia Incremental:** Debe ser incremental. Los dos esquemas principales son:
  - **Integración de arriba hacia abajo (*Top-down*):** Se comienza probando los módulos de control superior.
  - **Integración de abajo hacia arriba (*Bottom-up*):** Se comienzan probando los módulos inferiores.
- **Buena Práctica:** Los módulos críticos deben ser probados lo más tempranamente posible.

Niveles de Testing: Testing de Sistema

- **Síntesis:** Prueba realizada al **sistema completo**, buscando verificar que la aplicación funciona como un todo.
- **Alcance:** Investiga **requerimientos funcionales y no funcionales** del sistema.
- **Requerimientos No Funcionales (RNF):** Se prueban atributos como la recuperación de fallas, la seguridad, la protección, el rendimiento (*performance*) y el estrés.
- **Ambiente:** El entorno de prueba debe corresponder al **entorno de producción** tanto como sea posible para minimizar el riesgo de incidentes específicos del ambiente.

Niveles de Testing: Testing de Aceptación

- **Síntesis:** Es la prueba realizada por el **usuario** o cliente para determinar si la aplicación se **ajusta a sus necesidades** (Validación).
- **Objetivo:** La meta principal es establecer **confianza en el sistema**. **Encontrar defectos no es el foco principal**.
- **Tipos de Pruebas de Usuario:**
  - **Pruebas Alfa:** Realizadas por el usuario en un ambiente de laboratorio (controlado).
  - **Pruebas Beta:** Pruebas en ambientes de trabajo reales (clientes potenciales usan el sistema y reportan problemas).

Ambientes

Los ambientes de testing son los recursos de *hardware* y *software* necesarios para trabajar con el producto. Se requieren distintos ambientes en el desarrollo de software:

- **Desarrollo:** Usado por los desarrolladores para construir y probar su código. Aquí se ejecutan pruebas unitarias y de integración. Se busca agilidad y retroalimentación rápida.
- **Pruebas (Testing):** Ambiente que utilizan los *testers* para llevar a cabo las pruebas de sistema. Los desarrolladores generalmente no deberían tener acceso.
- **Pre-Producción:** Ambiente que el cliente usa para probar el sistema en una instalación que debe ser **totalmente simétrica** a la de producción. Aquí se corren las pruebas de aceptación.
- **Producción:** Configuración final utilizada por los usuarios para su desempeño laboral. No se deben realizar pruebas en este entorno.

Proceso de Pruebas

El testing es un **proceso definido** que consta de etapas estructuradas.

1. **Planificación:** Determinar cómo se incluirá el testing en el plan del proyecto. Esto incluye definir los riesgos, los recursos, el criterio de aceptación y los entornos a emular.
2. **Identificación y Especificación:** Revisión de la base de pruebas (como los requerimientos) y la verificación de las especificaciones del *software*.
3. **Construcción del Test Plan:** Detalla los riesgos, objetivos del testing, estrategia, recursos y el criterio de aceptación.
4. **Ejecución:** Correr los Casos de Prueba (CP), registrar y comparar los resultados, y generar reportes de defectos. Se debe buscar la automatización para ahorrar tiempo y costo.
5. **Controlar/Evaluación (Cierre):** Revisar los resultados, evaluar el *test coverage* y el criterio de aceptación, y tomar decisiones. También incluye el análisis de las lecciones aprendidas.
6. **Riesgo:** Si se realiza un **Testing Ad Hoc** (sin proceso definido), se pierde la trazabilidad de lo que se probó.

Estrategias de Prueba

Las estrategias de prueba buscan optimizar el tiempo aplicado al *testing* limitando el número de casos de prueba a simular.

- **Enfoque de la Estrategia:** La prueba comienza en los componentes y opera "hacia afuera" hacia la integración de todo el sistema (siguiendo el Modelo en V).
- **Caja Negra (Black Box):** Se basa en los **requerimientos y la funcionalidad**. Se enfoca en las entradas y salidas del sistema, sin conocimiento de su estructura interna. Es ideal para pruebas de aceptación y sistema.
- **Caja Blanca (White Box):** Se basa en la **estructura interna** y la lógica del código. Requiere acceso al código fuente para verificar la lógica interna y las rutas de control.

Métodos de Prueba

Caja Blanca

Los métodos de Caja Blanca analizan la estructura interna del *software*. Permiten garantizar el *testing coverage*.

- **Complejidad Ciclomática:** Métrica que proporciona una medición cuantitativa de la complejidad de un programa. Se utiliza en el *testing* para definir el número de **caminos independientes** en el conjunto básico que deben ser probados.
- **Cobertura de Estructura:** Se enfoca en asegurar la ejecución de ciertos elementos estructurales. Ejemplos incluyen: Cobertura de enunciados, cobertura de decisión/caminos básicos, cobertura de condición, cobertura múltiple.

Caja Negra

Los métodos de Caja Negra se basan en las especificaciones.

- **Partición de Equivalencias:** Proceso sistemático que identifica un conjunto de **clases de pruebas representativas** de grandes conjuntos de otras pruebas posibles. Se identifican clases de equivalencia **válidas y no válidas**.
- **Análisis de Valores Límites (*Boundary Value Analysis*):** Se enfoca en los **límites** o valores de borde de las clases de equivalencia. Esto se debe a que los *bugs* suelen esconderse en los límites.
- **Basados en Gráficos:** Métodos que utilizan representaciones gráficas (como gráficos de flujo de datos o gráficos causa-efecto) para derivar casos de prueba.

Basados en la Experiencia

La experiencia del *tester* es fundamental para determinar las entradas del sistema y analizar los resultados.

- **Adivinanza de Defectos (*Defect Guessing*):** Enfoque basado en la **intuición y la experiencia** para identificar pruebas que probablemente expondrán defectos. Se elabora una lista de situaciones propensas a error (ej. *strings* nulos o vacíos, números negativos) y se crean pruebas a partir de ella.
- **Testing Exploratorio (*Exploratory Testing*):** El **aprendizaje, el diseño y la ejecución de las pruebas se realizan de forma simultánea**. La efectividad se apoya en el **conocimiento, habilidades y creatividad** del *tester*. Su foco está en encontrar problemas mediante la exploración del sistema bajo prueba.

Tipos de Prueba

Testing Funcional

Se basa en funciones y características del *software*.

- **Basado en Requerimientos:** Las pruebas se derivan directamente de los documentos de especificación (Requerimientos, Casos de Uso, *User Stories*).
- **Basado en los Procesos de Negocio:** Se prueba un proceso completo de negocio de principio a fin, por ejemplo, la secuencia completa de una transacción de venta.

Testing No Funcional

Es la prueba de **"cómo" funciona el sistema**, enfocándose en los requerimientos no funcionales (RNF), que son tan importantes como los funcionales.

- **Performance Testing (Pruebas de Rendimiento):** Pruebas de velocidad y eficiencia.
  - **Pruebas de Carga (*Load Testing*):** Simula la realidad a la que estará expuesto el sistema en producción.
  - **Pruebas de Estrés (*Stress Testing*):** Va más allá de la carga esperada para ver dónde se "rompe" el sistema.
- **Pruebas de Interfaces de Usuario (GUIs):** Las interfaces gráficas son mucho más complejas debido a que el usuario está en control y existe un mayor número de combinaciones posibles, lo que requiere más pruebas.
- **Otras Pruebas No Funcionales:** Seguridad, usabilidad, mantenimiento, fiabilidad y portabilidad.

Administración de Configuración

La Gestión de Configuración del Software (SCM) es una disciplina de soporte transversal a la Ingeniería de Software.

- **Vínculo con Testing:** Los **Casos de Prueba, los Guiones de Prueba, los Resultados y las Métricas de Calidad** son ítems de configuración de software (ICS) que deben ser versionados y mantenidos en un repositorio. SCM es crucial para establecer y mantener la integridad de los productos de trabajo. Esto es vital para las pruebas de regresión.

Secciones Transversales

Glosario Crítico

Concepto	Definición y Fuente	Vínculo
Testing	Proceso destructivo de ejecutar un programa con la intención de encontrar errores o defectos, cuya presencia se asume.	Función de Control de Calidad (QC).
Verificación (V)	Asegurar que se está construyendo el sistema correctamente (adherencia a especificaciones).	Se complementa con la Validación (V).
Validación (V)	Asegurar que se está construyendo el sistema correcto (cumplimiento de las necesidades del cliente).	Objetivo principal de las Pruebas de Aceptación.
Testing Temprano	Principio que establece que las pruebas deben comenzar lo antes posible (desde los requerimientos) para disminuir el costo de corrección de defectos.	Práctica clave en enfoques Ágiles.
Regresión	Ejecución de pruebas ya exitosas sobre un nuevo *build* del sistema para asegurar que los cambios no introdujeron nuevos defectos.	Habilitada y simplificada por la Automatización.
Partición de Equivalencias	Método de Caja Negra que identifica clases de entradas válidas y no válidas para reducir el número de casos de prueba necesarios.	Se utiliza junto al Análisis de Valores Límites.
TDD (*Test Driven Development*)	Técnica avanzada que consiste en escribir la prueba (generalmente unitaria) primero, luego el código funcional, y finalmente refactorizar.	Mejora la calidad del diseño y la cobertura de pruebas.
Smoke Test	Primera corrida de los *tests* de sistema que da cierto aseguramiento de que el *software* no provoca una falla catastrófica y está listo para pruebas formales.	Pruebas básicas que confirman la validez de la versión liberada a *testing*.

Cuadros Comparativos: Testing

Característica	Enfoque Tradicional (Definido)	Enfoque Ágil/Empírico
Proceso Base	Definido, lineal/secuencial (Modelo Cascada/V).	Empírico, iterativo/incremental (Scrum, XP).
Timing del Testing	Una fase separada al final del desarrollo.	Actividad **paralela y continua** al desarrollo. Se comienza lo antes posible.
Foco de la Prueba	Conformidad con las especificaciones detalladas (*up front*).	**Retroalimentación rápida** (*quick feedback*) y software funcional.
Testing Unitario	Realizado por desarrolladores; usualmente sin registro formal.	Incluido en la **Definición de Hecho (DoD)** de cada incremento. A menudo utiliza TDD.
Regresión	Ejecución manual costosa y lenta.	Automatizada y continua (por ejemplo, después de cada *commit* al repositorio).
Manejo de Roles	Roles bien separados (Desarrollador vs. Tester independiente).	Roles intercambiables; el equipo completo es responsable de la calidad.

Checklist de Buenas Prácticas y Riesgos

Concepto	Buenas Prácticas	Riesgos/Métricas
Diseño de Pruebas	Usar la **Partición de Equivalencias** y **Valores Límites** para optimizar el número de casos de prueba con alto potencial de encontrar fallas.	**Testing Exhaustivo es Imposible**. Se debe limitar la cantidad de casos a simular (ej. no más de 15 en pruebas de *performance*).
Calidad del Código	Adoptar **TDD** (escribir pruebas unitarias primero) para mejorar el diseño y la cobertura de las pruebas unitarias.	**Paradoja del Pesticida:** La confianza excesiva en pruebas automatizadas no cubre todas las funcionalidades si las pruebas no se renuevan.
Ambientes	Emular el entorno de producción en el ambiente de testing de sistema y de pre-producción.	Probar en el ambiente de producción tiene **grandes consecuencias** y no debería hacerse.

Concepto	Buenas Prácticas	Riesgos/Métricas
Gestión de Defectos	Establecer y documentar la **Severidad** (impacto) y la **Prioridad** (urgencia) para guiar la corrección.	**Latencia Alta:** Si un defecto detectado en *testing* tarda en corregirse, aumenta el costo de retrabajo.
Testing No Funcional	Priorizar las pruebas de carga, rendimiento y seguridad. Utilizar el conocimiento de los desarrolladores para identificar las funcionalidades más riesgosas o intensivas en recursos.	**Estrategia Tarde:** Hacer las pruebas de *performance* al final es como pedir un examen de sangre cuando el paciente ya está muerto.

Mapa de Relaciones Conceptuales

Unidad Relacionada	Concepto de Testing	Vínculo Detallado
Gestión de Proyectos	Criterios de Aceptación, Riesgo.	El **Criterio de Aceptación** es la herramienta clave para determinar cuándo una fase de testing ha finalizado. El testing se enfoca en **reducir riesgos** potenciales.
Calidad de Software (SQA/QC)	Verificación y Validación.	La **Verificación** busca el cumplimiento de estándares; la **Validación** busca el cumplimiento de las necesidades del cliente. El testing es parte del Control de Calidad.
Despliegue (DevOps/CI/CD)	Automatización, Regresión, CI.	La **Automatización de Regresión** es fundamental para entornos de **Integración Continua (CI)** y **Continuous Delivery**, ya que permite que el código sea testeado después de cada cambio y esté potencialmente desplegable.
Ingeniería de Requerimientos	Testing Temprano, Caja Negra.	El testing comienza con los requerimientos (Testing Temprano) para asegurar que son testeables y legibles. Los métodos de **Caja Negra** (ej. Partición de Equivalencias) se basan directamente en las especificaciones funcionales.
Administración de Configuración	Artefactos de Prueba, Control de Versión.	Todos los artefactos de testing (casos, scripts, resultados) deben ser ítems de configuración bajo **SCM**, asegurando su trazabilidad e integridad.